

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

QUESTIONS:5 Total Marks 20 Duration :60 Minutes Annexure A
Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I M.Venkata Veeranajneya Reddy (192472140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.Venkata Veeranajneya Reddy

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	

Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline

A system must process a live video stream to:

- A. Capture frames from a camera
- B. Detect faces in each frame
- C. Recognize faces using a pre-trained dataset
- D. Display bounding boxes and labels in real time

Constraints:

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

Ans:

Requirements: Capture live frames, detect faces, recognize faces using a pre-trained dataset, and display bounding boxes and labels. It must operate continuously, minimize delay, and handle multiple faces.

Pseudocode

ALGORITHM RealTimeFaceRecognition

START

LOAD face detection model

LOAD pre-trained face recognition dataset/model

INITIALIZE camera using OpenCV VideoCapture

WHILE camera is available DO

CAPTURE frame from camera

```
IF frame is empty THEN
    CONTINUE
END IF

RESIZE frame to suitable resolution

CONVERT frame to grayscale

DETECT all faces using face detector

FOR each detected face DO

    EXTRACT face region using bounding box

    PREPROCESS face region
        RESIZE face to required size
        NORMALIZE face if required

    RECOGNIZE face using pre-trained recognition model

    IF recognition confidence is above threshold THEN
        ASSIGN recognized person's name
    ELSE
        ASSIGN label = "Unknown"
    END IF

    DRAW bounding box around face

    DISPLAY person's name and confidence
    using cv2.rectangle() and cv2.putText()

END FOR

DISPLAY processed frame

IF user presses 'q' THEN
    BREAK
END IF

END WHILE

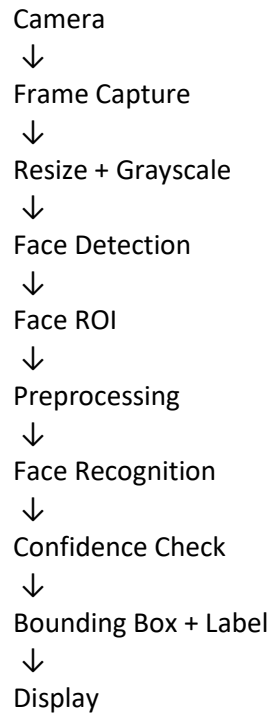
RELEASE camera
CLOSE all OpenCV windows

END
```

Why this satisfies the constraints

- **Continuous operation:** WHILE loop continuously processes camera frames.
- **Minimum delay:** Resize frames and process only required face regions.
- **Multiple faces:** FOR each detected face processes every face independently.
- **False recognition:** Confidence threshold identifies uncertain faces as **Unknown**.
- **Real-time display:** Bounding boxes and labels are drawn immediately.

Pipeline:



2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
- B. Enhance image quality (noise removal, contrast improvement)
- C. Segment text regions
- D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

Ans:

The system must read printed-text images, improve image quality, segment text regions, and extract/display recognized text. The input may be noisy and have low contrast, and **batch processing** is required.

Pseudocode

ALGORITHM OCRDocumentProcessing

START

GET list of input document images

FOR each image in input list DO

 READ image using cv2.imread()

 IF image cannot be read THEN

 DISPLAY "Invalid image"

 CONTINUE

 END IF

 CONVERT image to grayscale

 APPLY Gaussian Blur
 to reduce noise

 IMPROVE contrast
 using histogram equalization or CLAHE

 APPLY thresholding
 to separate text from background

 APPLY morphological operations
 to remove noise
 and connect broken text regions

 FIND contours using cv2.findContours()

 IDENTIFY text regions
 using contour size and shape

 SORT text regions
 from top to bottom
 and left to right

 FOR each text region DO

 CROP text region

 RESIZE region if necessary

SEND region to OCR/text recognition module

EXTRACT recognized text

STORE recognized text

END FOR

COMBINE all recognized text

DISPLAY extracted text

SAVE extracted text if required

END FOR

END

Why this satisfies the constraints

- **Noisy images:** Gaussian blur and morphological filtering reduce noise.
- **Low contrast:** Histogram equalization/CLAHE improves contrast.
- **Text segmentation:** Thresholding and contours separate text regions.
- **Batch processing:** FOR each image processes all input images.
- **Correct reading order:** Text regions are sorted before recognition.

Pipeline:

Input Image



Grayscale



Noise Removal



Contrast Enhancement



Thresholding



Morphological Filtering



Contour Detection



Text Region Segmentation



OCR



Recognized Text



Display / Save

3. Motion Detection and Tracking System

A surveillance system must:

- A. Read video input
- B. Detect motion between consecutive frames
- C. Track moving objects
- D. Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

Ans:

The surveillance system must read video, detect motion between consecutive frames, track moving objects, and highlight motion regions. It must handle **dynamic backgrounds** and operate in **near real-time**.

Pseudocode

ALGORITHM MotionDetectionAndTracking

START

 INITIALIZE video capture

 INITIALIZE background subtraction model
 using MOG2

 INITIALIZE object tracker

 READ first frame

 WHILE video is available DO

 READ current frame

 IF frame is empty THEN
 BREAK
 END IF

 RESIZE frame

 CONVERT frame to grayscale

 APPLY Gaussian Blur

```
GENERATE foreground mask
    using background subtraction

APPLY morphological opening
    to remove small noise

APPLY morphological closing
    to fill gaps in motion regions

FIND contours in foreground mask

INITIALIZE list of detected objects

FOR each contour DO

    CALCULATE contour area

    IF area is greater than minimum threshold THEN

        CALCULATE bounding rectangle

        CALCULATE object centroid

        ADD object to detected object list

    END IF

END FOR

UPDATE tracker
    using detected object positions

FOR each tracked object DO

    DRAW bounding box

    DRAW object ID

    DRAW centroid

END FOR

DISPLAY motion regions
DISPLAY processed frame
```

```
IF user presses 'q' THEN  
    BREAK  
END IF
```

```
END WHILE
```

```
RELEASE video  
CLOSE OpenCV windows
```

```
END
```

Why this satisfies the constraints

- **Dynamic background:** MOG2 continuously updates the background model.
- **Noise:** Gaussian blur and morphological operations remove unwanted changes.
- **Near real-time:** Lightweight frame processing and contour detection reduce delay.
- **Tracking:** Each detected moving object receives an ID and is followed across frames.
- **Visualization:** Bounding boxes and IDs highlight moving objects.

Pipeline:

Video Input



Frame Capture



Grayscale + Blur



MOG2 Background Subtraction



Morphological Filtering



Contour Detection



Motion Regions



Object Centroids



Tracking



Bounding Boxes + IDs



Display

4. Vehicle Counting System using Video

A traffic monitoring system must:

- A. Read video frames
- B. Detect vehicles entering a region of interest (ROI)
- C. Track movement across frames
- D. Count vehicles and display count

Constraints:

- Avoid double counting
- Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.

Ans:

The traffic monitoring system must read video frames, detect vehicles entering an ROI, track their movement, and count/display them. It must **avoid double counting** and handle **multiple vehicles simultaneously**.

Pseudocode

ALGORITHM VehicleCounting

START

 INITIALIZE video capture

 DEFINE Region Of Interest (ROI)

 DEFINE virtual counting line

 INITIALIZE vehicle detector

 INITIALIZE object tracker

 SET vehicle_count = 0

 SET counted_vehicle_IDs = empty list

 WHILE video is available DO

 READ frame

 IF frame is empty THEN

 BREAK

 END IF

 RESIZE frame

```
EXTRACT ROI from frame

DETECT vehicles inside ROI

FOR each detected vehicle DO

    CALCULATE bounding box

    CALCULATE vehicle centroid

END FOR

UPDATE tracker

FOR each tracked vehicle DO

    ASSIGN / UPDATE unique vehicle ID

    DRAW bounding box

    DRAW vehicle ID

    DRAW centroid

    IF vehicle centroid crosses counting line THEN

        IF vehicle ID is NOT in counted_vehicle_IDs THEN

            INCREMENT vehicle_count

            ADD vehicle ID
            to counted_vehicle_IDs

        END IF

    END IF

END FOR

DRAW ROI

DRAW counting line

DISPLAY vehicle_count
using cv2.putText()
```

DISPLAY processed frame

IF user presses 'q' THEN

BREAK

END IF

END WHILE

RELEASE video

CLOSE all windows

END

Why this satisfies the constraints

- **Avoid double counting:** Every vehicle receives a unique ID and is counted only if its ID is not already in counted_vehicle_IDs.
- **Multiple vehicles:** A FOR loop processes all detected vehicles in every frame.
- **ROI:** Processing only the required traffic region reduces unnecessary computation.
- **Tracking:** Vehicle positions are maintained across consecutive frames.
- **Visualization:** cv2.rectangle(), cv2.line(), and cv2.putText() display the tracking and count.

Pipeline:

Video



Frame Capture



ROI Selection



Vehicle Detection



Bounding Boxes



Centroid Calculation



Object Tracking + IDs



Counting Line Crossing



Check ID Already Counted?



Vehicle Count



Visualization

5. Facial Expression Recognition System

A mobile-based system must:

- A. Capture images from a camera
- B. Detect faces
- C. Extract facial features
- D. Classify expressions (happy, sad, neutral)
- E. Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

Ans:

The mobile-based system must capture camera images, detect faces, extract facial features, classify them as **happy, sad, or neutral**, and display the result. The constraints are **limited computational resources** and **real-time response**.

Pseudocode

ALGORITHM FacialExpressionRecognition

START

 INITIALIZE mobile camera

 LOAD lightweight face detection model

 LOAD pre-trained expression classification model

 WHILE camera is available DO

 CAPTURE frame

 IF frame is empty THEN

 CONTINUE

 END IF

 RESIZE frame

 to reduce computational cost

 CONVERT frame to grayscale

 APPLY lighting normalization

 DETECT faces in frame

 FOR each detected face DO

```

EXTRACT face ROI

RESIZE face ROI
    to required input size

EXTRACT facial features

CLASSIFY expression using
    pre-trained classifier

IF prediction = HAPPY THEN
    label = "Happy"

ELSE IF prediction = SAD THEN
    label = "Sad"

ELSE
    label = "Neutral"

END IF

DRAW bounding box
    around detected face

DISPLAY expression label
    using cv2.putText()

END FOR

DISPLAY processed frame

IF user presses 'q' THEN
    BREAK
END IF

END WHILE

RELEASE camera
CLOSE OpenCV windows

END

```

Why this satisfies the constraints

- **Limited computational resources:** Resize frames and use a lightweight face detector/classifier.
- **Real-time response:** Process low-resolution frames and only analyze detected face regions.

- **Efficient design:** Separate face detection, feature extraction, and classification into modules.
- **Expression classification:** The classifier produces exactly **Happy, Sad, or Neutral**.
- **Display:** Bounding boxes and expression labels are shown directly on the camera frame.

Pipeline:

Mobile Camera

↓

Frame Capture

↓

Resize

↓

Grayscale + Normalization

↓

Face Detection

↓

Face ROI

↓

Feature Extraction

↓

Expression Classification

↓

Happy / Sad / Neutral

↓

Bounding Box + Label

↓

Display

Very important exam pattern

For this assessment, write every answer in this structure:

ALGORITHM Name

START

INITIALIZE required models/camera

WHILE input is available DO

 READ frame/image

 PREPROCESS

 DETECT / SEGMENT

 FOR each detected object DO

 ANALYZE / TRACK / CLASSIFY

 END FOR

APPLY required condition

DRAW results

DISPLAY output

END WHILE

RELEASE resources

END

This matches the assessment rubric's emphasis on **complete algorithm design, loops, conditions, functions/modules, clear pseudocode structure, and handling all cases efficiently.**